

INFRASTRUCTURE CLOUD & RÉSEAU



FÉVRIER 2026

L'objectif principal de ce projet est de mettre en place une infrastructure d'authentification centralisée en utilisant GPG pour gérer les connexions SSH. Cette approche permet de remplacer l'agent SSH classique par une émulation gérée par l'agent GPG, offrant ainsi une gestion hiérarchique des identités plus sécurisée qu'une structure de clé plate. En s'appuyant sur des technologies modernes comme les courbes elliptiques ed25519, le système vise à renforcer la protection des accès serveurs tout en facilitant l'usage de supports matériels sécurisés comme les YubiKeys. Cette configuration garantit que les clés privées restent isolées et protégées par des mots de passe robustes et des dates d'expiration régulières, tout en restant totalement transparente pour les serveurs distants.

SSH over GPG une documentation complète et structurée pour mettre en place SSH via GPG, couvrant la théorie, la configuration technique et l'usage quotidien, du débutant au power-user, YubiKey incluse.

Avant de commencer, assurez-vous que votre environnement est compatible.

Versions recommandées

- GnuPG ≥ 2.2 (idéalement 2.4+)
- OpenSSH ≥ 7.2
- pinentry installé (pinentry-gtk, pinentry-curses, pinentry-mac...)

```
gpg --version
ssh -V
gpgconf --list-dirs
```

Nous devons faire cette vérification car `gpg` doit supporter l'émulateur SSH, aussi `ssh` doit supporter ed25519 et `gpgconf` permet de récupérer le bon socket agent

Création d'une clé GPG complète (si aucune clé existante)

```
gpg --expert --full-generate-key
```

Si vous n'avez pas encore de clé

Recommandation moderne :

- Type : ECC
C'est la **matière** (une technologie mathématique) de tes clés. Avant, les clés étaient énormes et lourdes (RSA). L'ECC, c'est comme une clé en métal du futur : elle est toute petite, mais elle est beaucoup plus solide.

- Signature : **ed25519**

C'est ta **clé de signature**, c'est cette clé que tu vas utiliser pour prouver ton identité quand tu te connectes à un serveur (SSH)

- Ajouter ensuite une sous-clé : **cv25519** (chiffrement)

C'est ta **clé de coffre-fort**. Elle ne sert pas à dire qui tu es, mais à fermer une boîte pour que personne ne puisse lire ce qu'il y a dedans, sauf toi. GPG la crée automatiquement pour que tu puisses recevoir des messages secrets.

- Expiration : 1–2 ans

C'est comme une **date de péremption** au bout de 2 ans, les clés s'autodétruisent et deviennent inutiles. Ça oblige à vérifier régulièrement que tu as toujours tes clés.

- Mot de passe fort

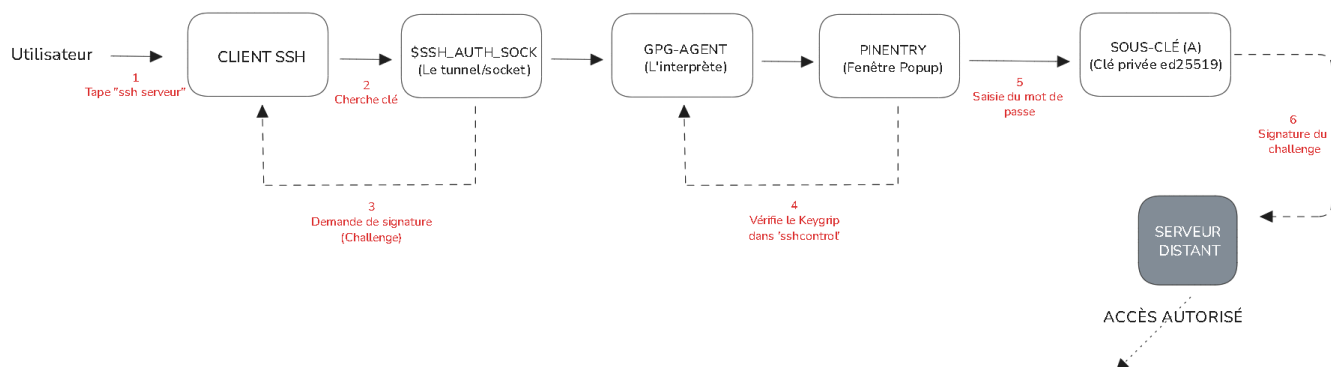
Tu crées une **identité** solide (ECC) qui peut à la fois prouver qui tu es (ed25519) et cacher tes secrets (cv25519), tout en étant protégée par un temps limité et un gros cadenas (mot de passe).

Lister vos clés

```
gpg --list-secret-keys --keyid-format=long
```

Architecture et Concept

Contrairement à `ssh-keygen`, GPG utilise un agent capable d'émuler SSH.



Le Socket (\$SSH_AUTH_SOCKET) → C'est la boîte aux lettres. SSH y dépose son courrier. Au lieu que ce soit `ssh-agent` qui l'ouvre, c'est `gpg-agent` qui a récupéré la clé de la boîte.

L'Agent GPG → Il reçoit une demande SSH mais il regarde dans ses dossiers GPG. Il ne traite la demande que si l'identifiant (**Keygrip**) de la clé est listé dans ton fichier `sshcontrol`.

La Sous-clé [A] → C'est le cœur du système. Elle reste verrouillée (chiffée) jusqu'à ce que `pinentry` (la popup) valide ton mot de passe.

Le Serveur → Lui, il ne voit que le résultat final, une signature mathématique parfaite. Il n'a aucune idée que tu as utilisé GPG pour la fabriquer.

Comment fonctionne réellement SSH over GPG

Quand vous faite

```
ssh user@server
```

SSH fait ceci

1. Cherche un socket via `SSH_AUTH_SOCK`
2. Envoie un challenge cryptographique
3. Demande à l'agent de signer ce challenge
4. Envoie la signature au serveur

Avec SSH classique :

```
ssh → ssh-agent → id_ed25519 → signature
```

Avec GPG :

```
ssh → gpg-agent (mode SSH) → sous-clé [A] → signature
```

GPG n'est pas utilisé directement par SSH et SSH parle à gpg-agent comme s'il parlait à ssh-agent.

gpg-agent implémente

```
The SSH Agent Protocol
```

C'est un protocole binaire documenté par OpenSSH

Quand vous activez

```
enable-ssh-support
```

gpg-agent :

- Ouvre un socket spécial
- Écoute les commandes `SSH_AGENTC_SIGN_REQUEST`

- Traduit la requête en signature OpenPGP
- Retourne une signature compatible SSH

Différence fondamentale : Clé SSH vs Clé GPG

Caractéristique	Clé SSH	Clé GPG
Format natif	OpenSSH	Clé maître + sous-clés spécialisées
Stockage	En clair ou chiffré sur disque	Keygrip interne, supporte smartcard native
Identité	Séparée	Web of Trust
Modèle	Clé plate	Hiérarchique
Point clé	Utilisée pour authentification réseau (SSH)	Utilisée pour chiffrement, signature et authentification

Keygrip interne

- C'est un identifiant unique interne utilisé par GPG pour repérer une clé sur le système, indépendamment de son format ou de son stockage.
- **Rôle** : Permet à GPG de gérer les clés de façon transparente, notamment pour l'association à des sous-clés ou à des smartcards.

C'est un hash interne représentant la clé publique dans libgcrypt

```
sshcontrol → keygrip
```

Dit à gpg-agent :

```
"Expose cette clé en mode SSH"
```

Supporte Smartcard native

- GPG peut stocker les clés directement sur une carte à puce (smartcard).
- **Avantage** : La clé privée ne quitte jamais la carte, ce qui renforce la sécurité et permet de signer ou déchiffrer sans exposer la clé sur le disque.

Clé maître + sous-clés spécialisées

- **Clé maître** : La clé principale qui contrôle l'identité et la confiance dans le Web of Trust.
 - **Sous-clés spécialisées** : Dérivées de la clé maître pour des usages précis :
 - Signature
 - Chiffrement
 - Authentification
 - **Avantage** : Si une sous-clé est compromise, la clé maître reste sûre.
-

Web of Trust

- Un système de validation décentralisé des identités.
 - **Principe** : Les utilisateurs signent les clés des autres pour attester de leur authenticité, créant un réseau de confiance.
 - **Différence avec SSH** : SSH ne possède pas ce mécanisme, l'authenticité repose simplement sur la possession de la clé publique correspondante.
-

Hiérarchique (vs plate)

- **SSH** : Clé plate → Une clé unique sert pour tout.
- **GPG** : Clé hiérarchique → Une clé maître avec des sous-clés spécialisées pour différents usages.
- **Conséquence** : Plus flexible et sécurisée, car différents usages n'exposent pas la clé maître.

SSH repose sur une **clé unique** et simple, alors que GPG adopte une **structure hiérarchique** avec une clé maître et des sous-clés spécialisées pour différents usages.

cv25519 ne fonctionne pas pour SSH car SSH utilise `ed25519` pour les **signatures**, tandis que `cv25519` (basé sur X25519) est destiné à l'**échange de clés Diffie-Hellman**, et SSH nécessite uniquement des signatures, pas d'échanges DH.

YubiKey

Quand vous faites `keytocard` :

- La clé privée quitte ton disque
- Elle est stockée dans un Secure Element
- Impossible à extraire (même par root)

La signature devient :

ssh → gpg-agent → smartcard → signature interne

Surface d'attaque

Même avec GPG, si l'agent est déverrouillé :

- Malware peut demander des signatures
- Serveur distant via agent forwarding peut signer

GPG protège la **clé privée**, pas l'agent en mémoire.

Protection réelle :

- TTL court
- Smartcard
- Touch confirmation

Clé GPG

```
gpg --expert --full-generate-key
```

```
gpg --expert --full-generate-key
[ NAWA ] test ~
> gpg --expert --full-generate-key
gpg (GnuPG) 2.4.7; Copyright (C) 2024 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Sélectionnez le type de clef désiré :
(1) RSA and RSA
(2) DSA and Elgamal
(3) DSA (sign only)
(4) RSA (sign only)
(7) DSA (set your own capabilities)
(8) RSA (set your own capabilities)
(9) ECC (sign and encrypt) *default*
(10) ECC (signature seule)
(11) ECC (set your own capabilities)
(13) Existing key
(14) Existing key from card
Quel est votre choix ? 9
Sélectionnez le type de courbe elliptique désiré :
(1) Curve 25519 *default*
(2) Curve 448
(3) NIST P-256
(4) NIST P-384
(5) NIST P-521
(6) Brainpool P-256
(7) Brainpool P-384
(8) Brainpool P-512
(9) secp256k1
Quel est votre choix ? 1
```

```

Veuillez indiquer le temps pendant lequel cette clef devrait être valable.
  0 = la clef n'expire pas
<n> = la clef expire dans n jours
<n>w = la clef expire dans n semaines
<n>m = la clef expire dans n mois
<n>y = la clef expire dans n ans
Pendant combien de temps la clef est-elle valable ? (0) 2 y
valeur incorrecte
Pendant combien de temps la clef est-elle valable ? (0) 2y
La clef expire le jeu. 17 févr. 2028 13:45:07 CET
Est-ce correct ? (o/N) o

GnuPG doit construire une identité pour identifier la clef.

Nom réel : nawa
Adresse électronique : aws867240@gmail.com
Commentaire :
Vous avez sélectionné cette identité :
  « nawa <aws867240@gmail.com> »

Changer le (N)om, le (C)ommentaire, l'(A)dresse électronique
ou (O)ui/(Q)uitter ? █

```

la clef expire dans n jours
la clef expire dans n semaines
la clef expire dans n mois
la clef e
len de ter
recte
len de ter
re le jeu
t ? (o/N)
onstruire
awa
tronique
lectionné
aws867240
(N)om, le
uitter ? o
octets ale
taper au
énération de nombres premiers ; cela donne au générateur de
toires une meilleure chance d'obtenir suffisamment d'entropie.

Phrase secrète :

Veuillez entrer la phrase secrète
pour protéger la nouvelle clef

```
gpg --expert --edit-key ton.email@exemple.com
```

Lance la création de la sous-clé. À l'invite `gpg>`, tape

```
addkey
```

```
[ NAWA ] test ~
> > gpg --expert --edit-key aws867240@gmail.com
gpg (GnuPG) 2.4.7; Copyright (C) 2024 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

La clef secrète est disponible.

gpg: vérification de la base de confiance
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: profondeur : 0  valables : 1  signées : 0
    confiance : 0 i., 0 n.d., 0 j., 0 m., 0 t., 1 u.
gpg: la prochaine vérification de la base de confiance aura lieu le 2028-02-17
sec  ed25519/0722B8676F15E108
    créé : 2026-02-17  expire : 2028-02-17  utilisation : SC
    confiance : ultime          validité : ultime
ssb  cv25519/D0630093926C4B18
    créé : 2026-02-17  expire : 2028-02-17  utilisation : E
[  ultime ] (1). nawa <aws867240@gmail.com>

gpg> addkey
Sélectionnez le type de clef désiré :
(3) DSA (sign only)
(4) RSA (sign only)
(5) Elgamal (encrypt only)
(6) RSA (encrypt only)
(7) DSA (set your own capabilities)
(8) RSA (set your own capabilities)
(10) ECC (signature seule)
(11) ECC (set your own capabilities)
(12) ECC (encrypt only)
(13) Existing key
(14) Existing key from card
Quel est votre choix ? 11
```

Faire la même étape

Possible actions for this ECC key: Signer Authenticifier
Actions actuellement permises : Signer

- (S) Inverser la capacité de signature
- (A) Inverser la capacité d'authentification
- (Q) Terminé

Quel est votre choix ? S

Possible actions for this ECC key: Signer Authenticifier
Actions actuellement permises :

- (S) Inverser la capacité de signature
- (A) Inverser la capacité d'authentification
- (Q) Terminé

Quel est votre choix ? A

Possible actions for this ECC key: Signer Authenticifier
Actions actuellement permises : Authenticifier

- (S) Inverser la capacité de signature
- (A) Inverser la capacité d'authentification
- (Q) Terminé

Quel est votre choix ? Q

Sélectionnez le type de courbe elliptique désiré :

- (1) Curve 25519 *default*
- (2) Curve 448
- (3) NIST P-256
- (4) NIST P-384
- (5) NIST P-521
- (6) Brainpool P-256
- (7) Brainpool P-384
- (8) Brainpool P-512
- (9) secp256k1

Quel est votre choix ? 1

Sauvegarder

save

Sans cela, tout ton travail sera perdu

gpg --list-keys --with-keygrip

```

[ NAWA ] test ~
> > gpg --list-keys --with-keygrip
/home/nawa/.gnupg/pubring.kbx
-----
pub   ed25519 2026-02-17 [SC] [expire : 2028-02-17]
      A9D4A2EB9C2D78644D2A178E0722B8676F15E108
      Keygrip = 8113E8606133C3647B26951A37674CF1EC89B196
uid    [   ultime ] nawa <aws867240@gmail.com>
sub    cv25519 2026-02-17 [E] [expire : 2028-02-17]
      Keygrip = C20D639A9726B493447A3A0F7393C23405C21AF7
sub    ed25519 2026-02-17 [A] [expire : 2028-02-17]
      Keygrip = AA2DE5F9BE2043E0AF28E309D329EABCBC09D6EA

[ NAWA ] test ~
> > echo "AA2DE5F9BE2043E0AF28E309D329EABCBC09D6EA" >> ~/.gnupg/sshcontrol
[ NAWA ] test ~
> > █

```

Regarde sous la ligne qui finit par [A] et copie la longue suite de chiffres/lettres

Ajoute le au contrôle SSH

```
echo "COLLE_TON_KEYGRIP_ICI" >> ~/.gnupg/sshcontrol
```

Pour que GPG prenne en compte ce changement immédiatement, demande-lui de se rafraîchir

```
gpg-connect-agent updatestartuptty /bye
```

Ouvrir nano pour

```
nano ~/.zshrc
```

Configuration GPG pour SSH

```

export GPG_TTY=$(tty)

# Dit à SSH de parler à GPG
export SSH_AUTH_SOCK=$(gpgconf --list-dirs agent-ssh-socket)

# Prépare l'affichage du mot de passe
gpg-connect-agent updatestartuptty /bye > /dev/null

```

Descends tout en bas du fichier et colle ces lignes


```

GNU nano 8.4 /home/nawa/.zshrc *
AG_HOST="%F{#B6FFDA}" # Foreground (Neutral Machine)
AG_PATH="%F{#C6FF00}" # Yellow (Location High-Viz)
AG_ARROW="%F{#00FF7B}" # Cursor Green (Action)
AG_GIT_CLEAN="%F{#00FF66}" # Green
AG_GIT_DIRTY="%F{#FF3355}" # Red
AG_RESET="%f"

# Git info logic
antigravity_git() {
    git rev-parse --is-inside-work-tree &>/dev/null || return
    local branch dirty
    branch=$(git symbolic-ref --short HEAD 2>/dev/null)
    git diff --quiet || dirty="*"
    if [[ -n $dirty ]]; then
        echo " ${AG_GIT_DIRTY}$branch$dirty${AG_RESET}"
    else
        echo " ${AG_GIT_CLEAN}$branch ✓${AG_RESET}"
    fi
}

# Prompt principal (2-line layout)
# Ligne 1: [ USER ] Machine ~ Git
# Ligne 2: > > (Vert si OK, Rouge si Erreur)
PROMPT='${AG_BRACKET}[ ${AG_USER}${USER:u} ${AG_BRACKET}] ${AG_HOST}%m ${AG_PATH}%~${AG_RESET}
%{?.${AG_ARROW}.${AG_GIT_DIRTY}}> > ${AG_RESET}'

# Load P10k if it exists
[[ ! -f ~/.p10k.zsh ]] || source ~/.p10k.zsh

export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh" # This loads nvm
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion" # This loads nvm bash_c

# Configuration GPG pour SSH
export GPG_TTY=$(tty)
export SSH_AUTH_SOCK=$(gpgconf --list-dirs agent-ssh-socket)
gpg-connect-agent updatestartuptty /bye > /dev/null

^G Aide      ^O Écrire    ^F Chercher  ^K Couper    ^T Exécuter  ^C Emplacement
^X Quitter   ^R Lire fich.^_ Remplacer  ^U Coller    ^J Justifier ^/ Aller ligne

```

```
source ~/.zshrc
```

Une fois que tu as fait le `source`, ta clé GPG devrait enfin apparaître


```
admin@ip-172-31-9-52: ~
pub  rsa4096 2026-02-19 [SC] [expire : 2028-02-19]
3CD58B03713418880E5DAB89FDFE6CA39BF1F9F6
Keygrip = 57A2DAC41BEC95F7E0A952A5C6D196FB84439599
uid  [  ultime ] nawa <aws867240@gmail.com>
sub  rsa4096 2026-02-19 [E] [expire : 2028-02-19]
Keygrip = A3270AB07CF7DD2783D7910F83E7DB644DFF641D
sub  ed25519 2026-02-19 [A] [expire : 2028-02-19]
Keygrip = DD62EDA9C4AAB0E0D4A8F4B56BDCB53B71521F50

[ NAWA ] test ~
> > echo "DD62EDA9C4AAB0E0D4A8F4B56BDCB53B71521F50" >> ~/.gnupg/sshcon
trol

[ NAWA ] test ~
> > gpg-connect-agent updatestartuptty /bye
OK
[ NAWA ] test ~
> > export SSH_AUTH_SOCK=$(gpgconf --list-dirs agent-ssh-socket)
[ NAWA ] test ~
> > ssh-add -L
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIB0JLxX8ws/cgo1K+Ujxcgr/vvvt9QUGE7
m1QH1MBT03 (none)
[ NAWA ] test ~
> >

[ NAWA ] test ~
> > ssh admin@13.53.167.140
The authenticity of host '13.53.167.140 (13.53.167.140)' can't be esta
blished.
ED25519 key fingerprint is SHA256:x6SDsiQ+3Ae0v2P5jvWdA6UXQLoX2vEe2vzb
crLPU5U.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
es
Warning: Permanently added '13.53.167.140' (ED25519) to the list of kn
own hosts.
Linux ip-172-31-9-52 6.12.48+deb13-cloud-amd64 #1 SMP PREEMPT_DYNAMIC
Debian 6.12.48-1 (2025-09-20) x86_64

The programs included with the Debian GNU/Linux system are free softwa
re;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Feb 19 09:01:50 2026 from 92.88.171.68
admin@ip-172-31-9-52:~$
```

```
export SSH_AUTH_SOCK=$(gpgconf --list-dirs agent-ssh-socket)
```

Pour que la commande `ssh` sache qu'elle doit parler à GPG, tu dois définir la variable d'environnement dans ton terminal Debian

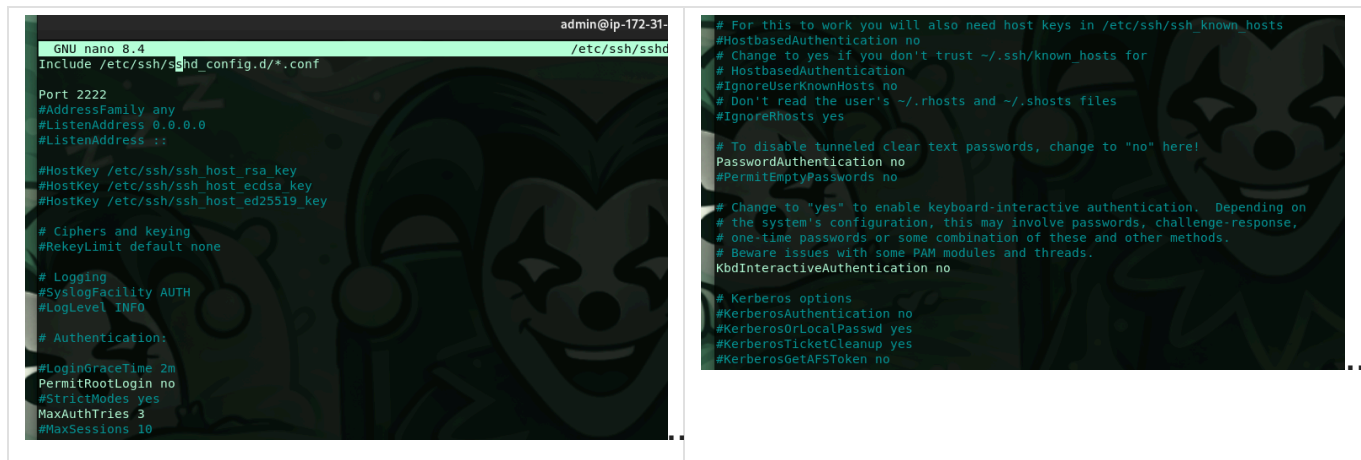
Tape cette commande pour voir si ta clé GPG est maintenant prête à être utilisée

```
ssh-add -L
```

Hardening du Système & Accès SSH

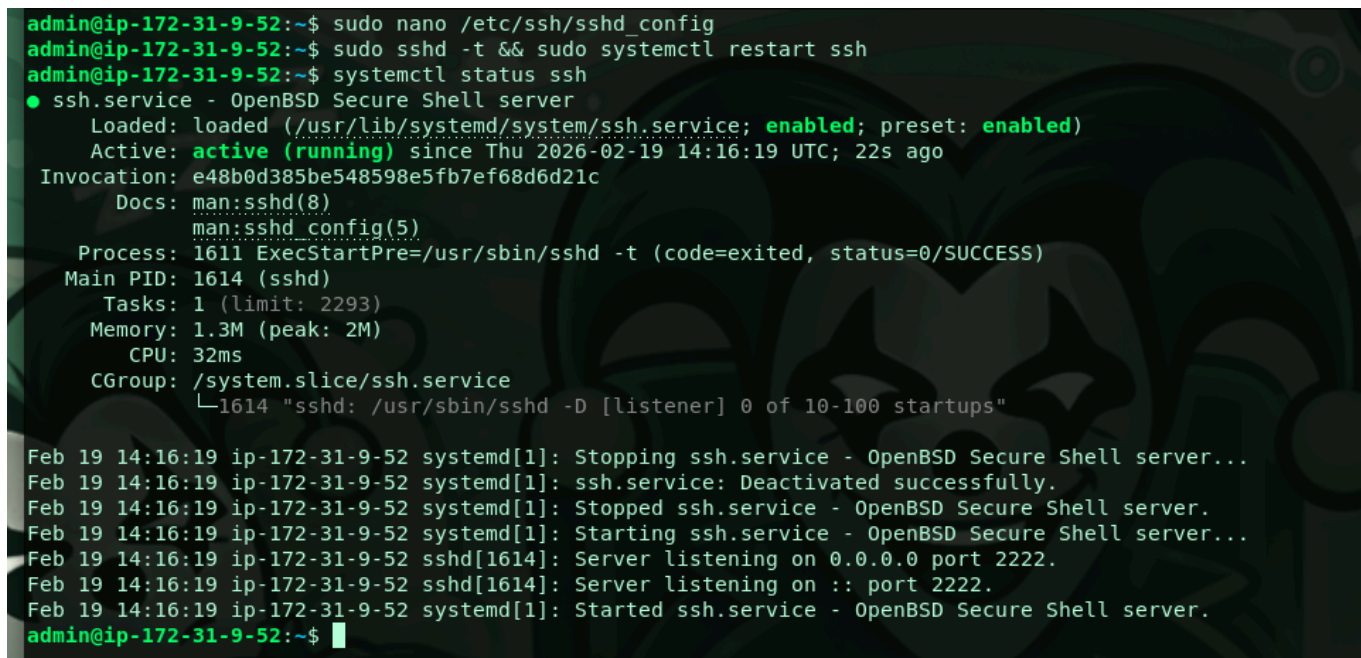
Applique ces configurations de sécurité dans le fichier `/etc/ssh/sshd_config`

Paramètre	Valeur conseillée	Pourquoi ?
Port	2222	Évite 99% des scans automatiques de bots.
PermitRootLogin	no	Empêche une attaque directe sur le compte root.
PasswordAuthentication	no	Force l'usage de ta clé GPG (incassable par brute-force).
MaxAuthTries	3	Coupe la connexion après 3 échecs.
AllowUsers	admin	Seul ton utilisateur a le droit de tenter une connexion.



Commande pour appliquer les changement

```
sudo sshd -t && sudo systemctl restart ssh
```



Ne ferme pas cette console

Avant de quitter, tu dois tester ta nouvelle connexion depuis un **autre terminal** sur ta machine locale. Si tu as oublié d'ouvrir le port dans le Security Group AWS, tu ne pourras plus rentrer.

Ouvre un nouveau terminal sur ta VM Debian et teste :

```
ssh -p 2222 admin@13.53.167.140
```

Si ça ne fonctionne pas, suive cette étape

- Va dans **EC2 > Instances** > Clique sur ton instance.
- Onglet **Sécurité** > Clique sur le nom du **Security Group**.
- Clique sur **Edit inbound rules** (Modifier les règles entrantes).
- Ajoute une règle :
 - **Type** : Custom TCP.
 - **Port range** : 2222.
 - **Source** : 0.0.0.0/0 (pour tester) ou My IP (pour la sécurité).
- Sauvegarde la règle.

The screenshot shows the 'Règles entrantes' (Inbound Rules) page in the AWS IAM console. It displays a table with one rule. The rule has the ID 'sgr-09091fb305ae7868b', Type 'SSH', Protocol 'TCP', Port Range '22', Source 'Personal IP', and Description 'SSH'. Below the table is an 'Ajouter une règle' button. At the bottom right are 'Annuler', 'Aperçu des modifications', and 'Enregistrer les règles' buttons.

- **Modifie la règle existante** ou clique sur **Ajouter une règle**.
- **Type** : Sélectionne TCP personnalisé (au lieu de SSH).
- **Plage de ports** : Remplace 22 par 2222 .
- **Source** : Garde ton IP actuelle pour que toi seul puisses entrer.
- **Enregistre les règles** (bouton orange en bas à droite).

Une fois la règle enregistrée sur AWS, retourne dans ton terminal Debian et relance la commande

```
ssh -v -p 2222 admin@13.53.167.140
```

Le `-v` est utile ici : si ça bloque encore, il nous dira si c'est un problème de réseau ou d'authentification GPG

Automatisation des mises à jour pour valider ton plan de hardening

```
sudo apt update && sudo apt install unattended-upgrades -y
sudo dpkg-reconfigure -plow unattended-upgrades
```

Sélectionne "Oui" pour activer le téléchargement et l'installation automatique des correctifs de sécurité

Vérifie si les mises à jour automatiques sont déjà là

```
sudo unattended-upgrades --dry-run --debug
```

Une fois que le test est fini, tu peux vérifier que le service est bien "activé" au démarrage avec

```
systemctl status unattended-upgrades
```

```
admin@ip-172-31-9-52: ~
admin@ip-172-31-9-52:~$ systemctl status unattended-upgrades
● unattended-upgrades.service - Unattended Upgrades Shutdown
   Loaded: loaded (/usr/lib/systemd/system/unattended-upgrades.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-02-19 08:51:57 UTC; 5h 47min ago
 Invocation: 30bfae3bd2df4671b6e32255b13629af
    Docs: man:unattended-upgrade(8)
   Main PID: 792 (unattended-upgr)
      Tasks: 1 (limit: 2293)
    Memory: 12.6M (peak: 14.2M)
         CPU: 1.911s
   CGroup: /system.slice/unattended-upgrades.service
           └─792 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal

Feb 19 08:51:57 ip-172-31-9-52 systemd[1]: Started unattended-upgrades.service - Unattended Upgrades Shutdown.
admin@ip-172-31-9-52:~$
```

Durcissement du Noyau (Sysctl)

Crée le fichier de configuration

```
sudo nano /etc/sysctl.d/99-hardening.conf
```

Contenu à copier/coller

```
# --- PROTECTION MÉMOIRE & PROCESSUS ---
# Restreint l'accès aux pointeurs noyau (Anti-Exploit)
kernel.kptr_restrict = 2
# Restreint l'accès au buffer dmesg (Info Leak)
kernel.dmesg_restrict = 1
# Désactive l'eBPF pour les utilisateurs non privilégiés
kernel.unprivileged_bpf_disabled = 1
# Durcit le compilateur JIT BPF
net.core.bpf_jit_harden = 2
# Randomisation complète de l'espace mémoire (ASLR)
kernel.randomize_va_space = 2
# Empêche le débogage de processus par d'autres (ptrace)
kernel.yama.ptrace_scope = 2

# --- DURCISSEMENT RÉSEAU (Anti-Spoofing & DoS) ---
# Désactive l'IP Forwarding (Sauf si tu deviens un routeur plus tard)
net.ipv4.ip_forward = 0
```

```
# Protection contre les attaques par inondation SYN (SYN Cookies)
net.ipv4.tcp_syncookies = 1
# Ignore les redirections ICMP (Anti-MITM)
net.ipv4.conf.all.accept_redirects = 0
net.ipv4.conf.default.accept_redirects = 0
net.ipv6.conf.all.accept_redirects = 0
# Ignore les réponses aux pings (Mode furtif optionnel)
# net.ipv4.icmp_echo_ignore_all = 1
```

Application immédiate

```
sudo sysctl -p /etc/sysctl.d/99-hardening.conf
```

Politiques AppArmor (Confinement)

AppArmor est déjà installé par défaut sur Debian, mais il faut s'assurer qu'il est bien actif et qu'il confine le maximum de services.

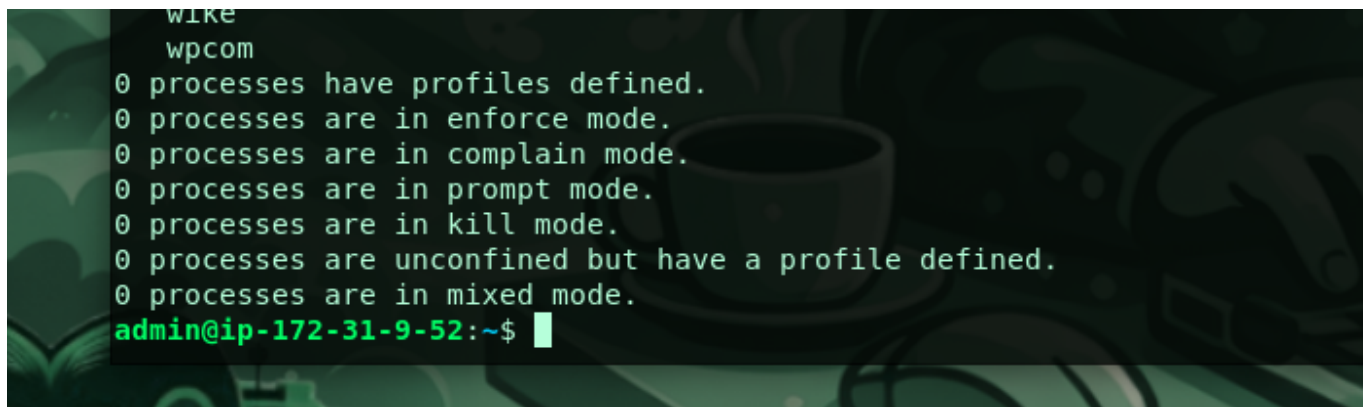
Installe les profils supplémentaires et les outils d'audit

```
sudo apt install apparmor-profiles apparmor-profiles-extra apparmor-utils -y
```

Vérifie l'état actuel

```
sudo aa-status
```

Tu devrais voir une liste de profils en mode "enforce" (actif) ou "complain" (audit uniquement

A terminal window with a dark green background and a faint illustration of a coffee cup and books. The terminal output shows the status of AppArmor profiles. The first two lines are partially cut off: 'wike' and 'wpcor'. The output lists the number of processes in various modes: 0 processes have profiles defined, 0 in enforce mode, 0 in complain mode, 0 in prompt mode, 0 in kill mode, 0 unconfined but with a profile, and 0 in mixed mode. The prompt is 'admin@ip-172-31-9-52:~\$'.

Active tous les profils disponibles en mode restrictif

```
sudo aa-enforce /etc/apparmor.d/*
```

Vérification de la conformité

Pour t'assurer que tout est bien en place avant d'aller dormir ou de passer à la suite

```
sudo /sbin/sysctl kernel.randomize_va_space
```

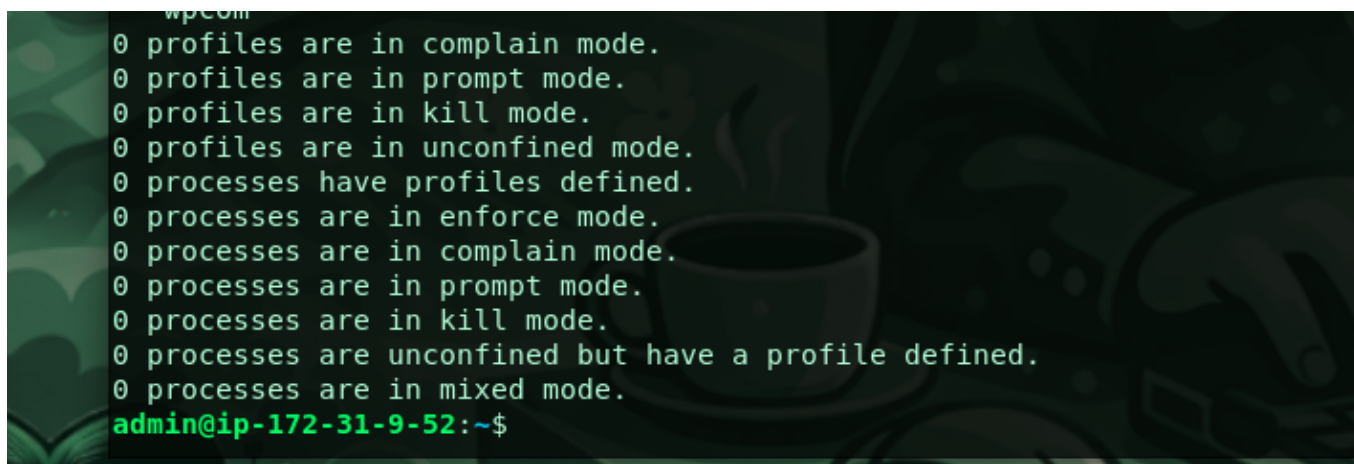
Doit renvoyer 2

Vérifie si AppArmor est actif

```
sudo /usr/sbin/aa-status
```

ou

```
sudo aa-status
```

A terminal window with a dark background and green text. The output of the 'aa-status' command is displayed, showing the status of various AppArmor profiles and processes. The text is as follows:

```
wpc@ip-172-31-9-52:~$ sudo aa-status
0 profiles are in complain mode.
0 profiles are in prompt mode.
0 profiles are in kill mode.
0 profiles are in unconfined mode.
0 processes have profiles defined.
0 processes are in enforce mode.
0 processes are in complain mode.
0 processes are in prompt mode.
0 processes are in kill mode.
0 processes are unconfined but have a profile defined.
0 processes are in mixed mode.
admin@ip-172-31-9-52:~$
```

Pour valider ton travail de hardening

- **Audit des services** : `systemctl --failed` (doit être vide).
- **Audit AppArmor** : `sudo /usr/sbin/aa-status` (doit lister des processus confinés).

Une fois ces étapes validées, ton serveur AWS respecte les standards de sécurité élevés décrits dans ton document.

```
admin@ip-172-31-9-52:~$ systemctl --failed
UNIT LOAD ACTIVE SUB DESCRIPTION
0 loaded units listed.
admin@ip-172-31-9-52:~$
```

```
smbldap-useradd
smbldap-useradd///etc/init.d/nscd
steam
stress-ng
surfshark
syslog-ng
syslogd
systemd-coredump
tcpdump
toybox
traceroute
transmission-cli
transmission-daemon
transmission-gtk
transmission-qt
trinity
tup
tuxedo-control-center
unix-chkpwd
unprivileged users
userbindmount
uwsgi-core
vdens
virtiofsd
vivaldi-bin
vpnn
vscode
wike
wpcrm
0 profiles are in complain mode.
0 profiles are in prompt mode.
0 profiles are in kill mode.
0 profiles are in unconfined mode.
0 processes have profiles defined.
0 processes are in enforce mode.
0 processes are in complain mode.
0 processes are in prompt mode.
0 processes are in kill mode.
0 processes are unconfined but have a profile defined.
0 processes are in mixed mode.
admin@ip-172-31-9-52:~$
```

Le Pont Sécurisé VPN WireGuard

Un VPN crée un **tunnel chiffré** entre ton appareil et un serveur distant, ce qui masque ton adresse IP et protège tes données en les faisant transiter de manière sécurisée sur Internet.

Quatre types fondamentaux de VPN

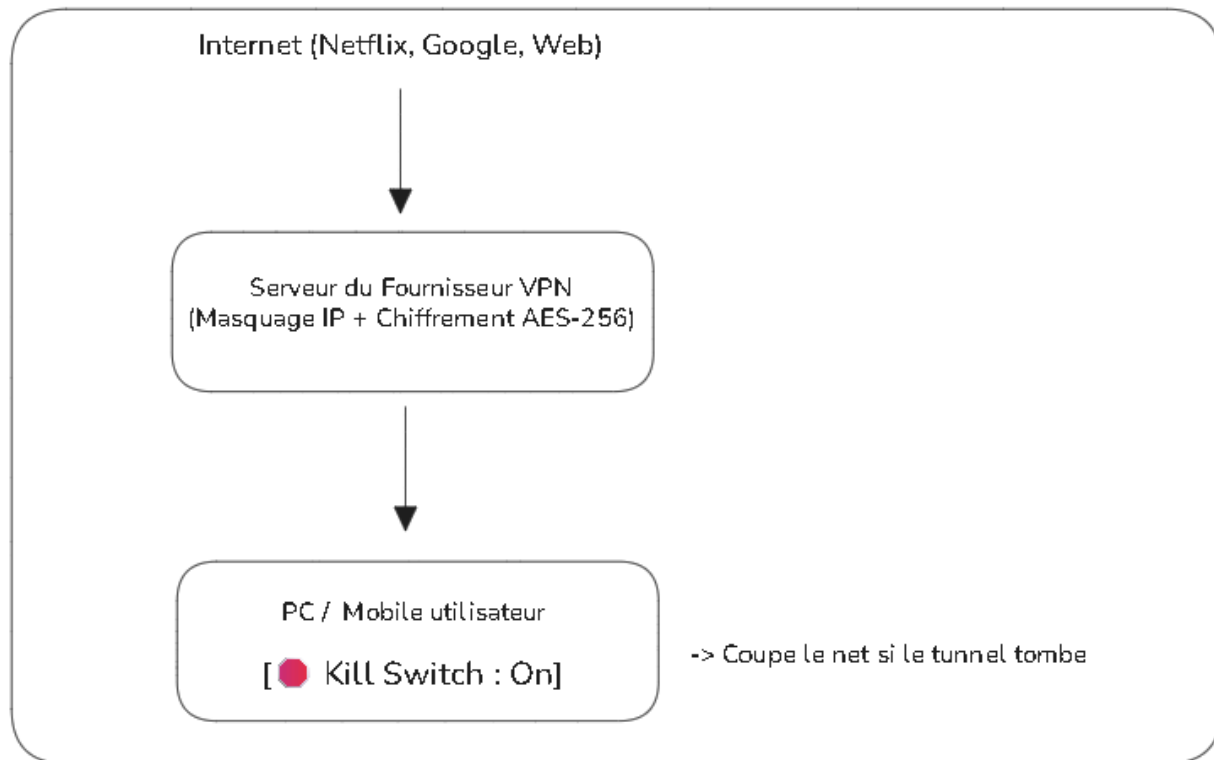
Ce qui définit le type de VPN utilisé est principalement la nature des **points de terminaison** (les appareils qui établissent la connexion sécurisée)

- **Services VPN commerciaux** : Ce sont des services par abonnement où le tunnel de chiffrement est établi entre votre appareil (PC, mobile) et un serveur maintenu par le fournisseur de VPN. Il sert principalement à sécuriser vos transactions en ligne et à masquer votre identité.

VPN Commercial (Usage Personnel)

Le bouclier pour la vie privée

Travaillant en Couche 3 (Réseau), il englobe tout le trafic de la machine



Objectif : Anonymat, contournement de la censure et du géo-blocage.

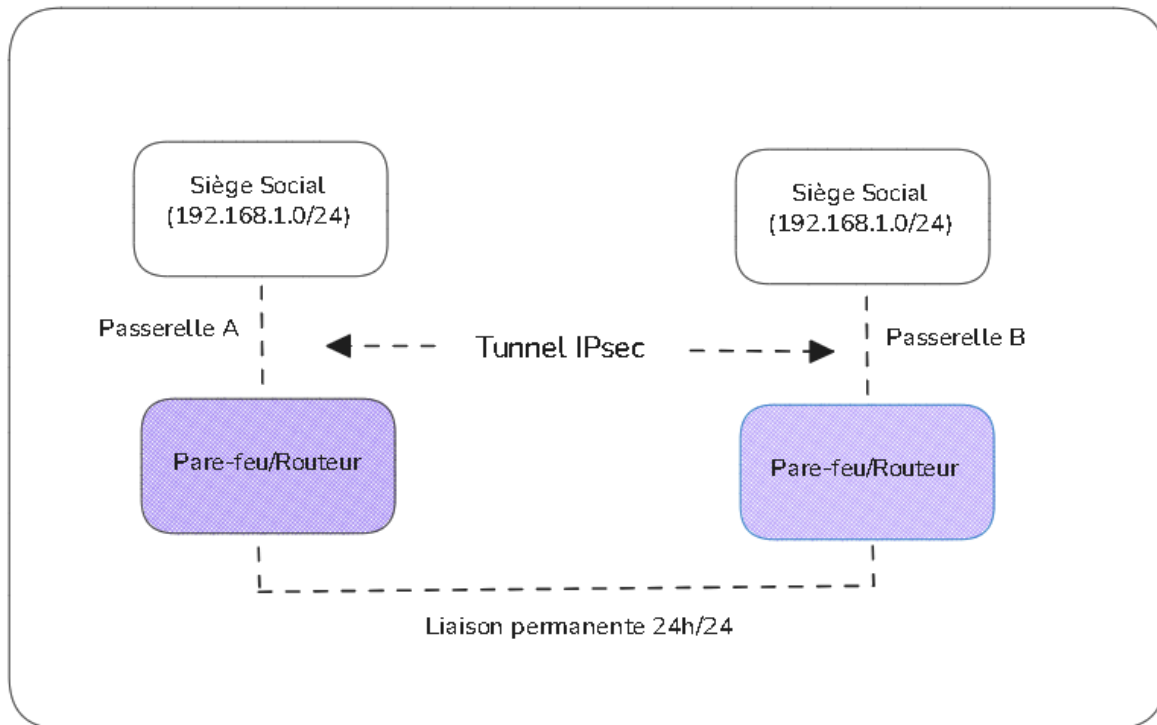
Option Multi-Hop : Le trafic rebondit sur deux serveurs (ex: Suisse -> Islande) pour un anonymat total

- **VPN de site à site (Site-to-site)** : Ces VPN utilisent des équipements réseau comme des **routeurs** ou des **pare-feux** comme points de terminaison. Ils permettent de relier deux réseaux distants (comme des bureaux de branche) pour qu'ils fonctionnent comme s'ils étaient sur le même réseau physique.

VPN Site-to-Site (Liaison Inter-sites)

Le pont permanent entre deux infrastructures

Utilisé pour relier physiquement deux routeurs de manière transparente



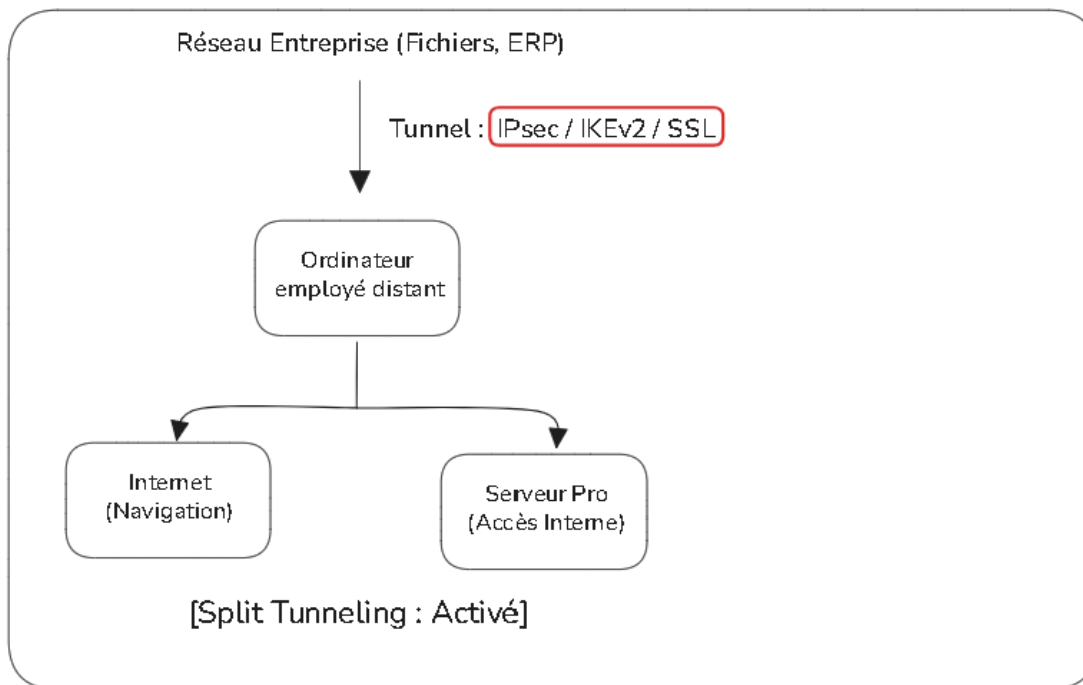
Transparence : Les employés de l'agence accèdent aux serveurs du siège comme s'ils étaient sur le même switch. Aucun logiciel n'est installé sur les PC.

- **VPN d'accès distant (Remote Access)** : Ici, le point de terminaison est un logiciel client sur l'appareil de l'utilisateur qui se connecte à un serveur VPN ou un routeur situé sur un site distant (généralement une entreprise).

VPN d'Accès Distant & ZTNA (Télétravail)

Le tunnel du collaborateur nomade

Évolution du VPN classique vers le Zero Trust Network Access



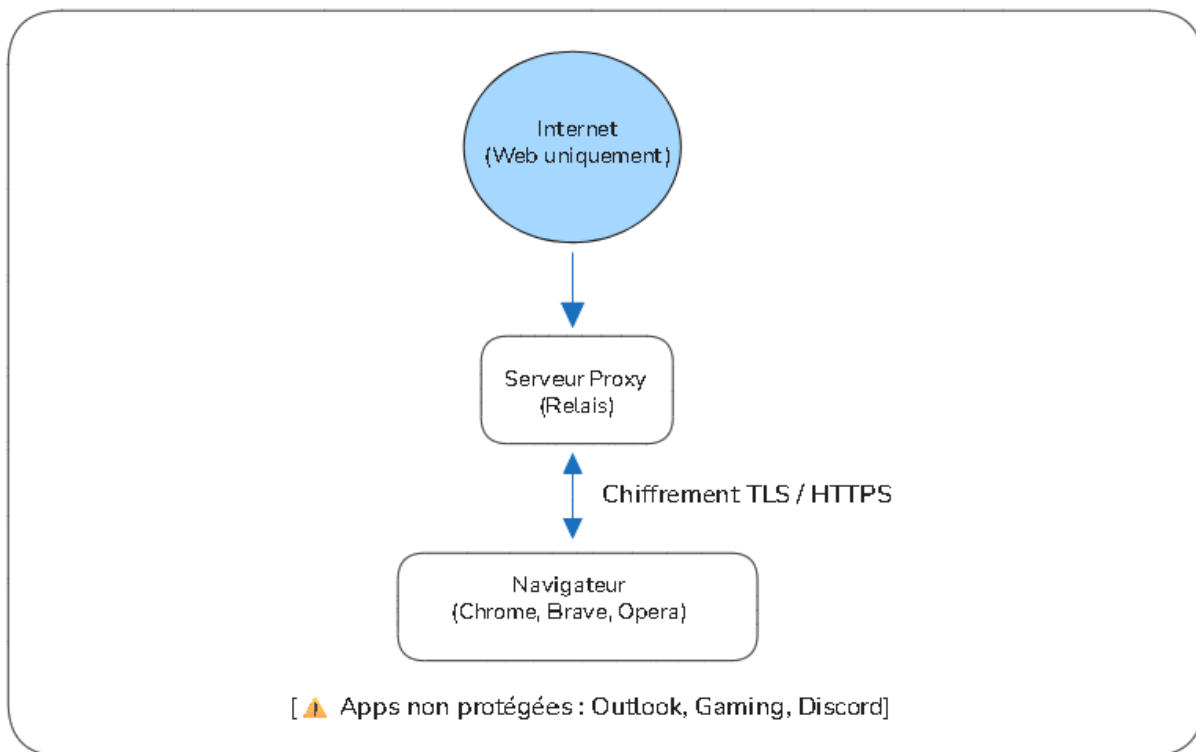
Nouveauté ZTNA : Contrairement au VPN qui donne accès à tout le réseau, le ZTNA vérifie l'identité à chaque clic et ne donne accès qu'à une application précise

- **VPN par navigateur :** Ce type est limité aux activités effectuées au sein d'un **navigateur web** spécifique. Il ne chiffre que le trafic web, souvent pour un site particulier (comme une banque via le protocole HTTPS/SSL).

Proxy de Navigateur (Souvent confondu avec un VPN)

La protection légère de session

Attention : Contrairement aux 3 précédents, il travaille en Couche 7 (Application)



Limitation : Ce n'est pas un tunnel système. Si le navigateur crash ou si vous utilisez une autre application, votre IP réelle est exposée.

Quand privilégier chaque type

Type de VPN	Quand l'utiliser	Objectif principal	Public concerné
VPN de site à site	Lorsque vous devez connecter deux réseaux entiers de manière permanente	Permettre à tous les appareils d'un bureau local de communiquer de façon sécurisée et transparente avec ceux d'un bureau distant, sans abonnement tiers	Entreprises avec plusieurs sites
VPN d'accès distant	Pour les travailleurs mobiles ou en télétravail	Permettre à un utilisateur individuel de se connecter au réseau de son entreprise depuis l'extérieur avec les mêmes privilèges qu'au bureau	Employés en télétravail

Type de VPN	Quand l'utiliser	Objectif principal	Public concerné
VPN commercial	Pour un usage personnel	Masquer l'activité de navigation au FAI et sécuriser la connexion sur un Wi-Fi public	Particuliers

Tableau Récapitulatif Technique

Type	Niveau OSI	Protocole Star	Portée
Commercial	Couche 3	WireGuard	Tout l'appareil
Accès Distant	Couche 3/4	AnyConnect / IPsec	Réseau Pro spécifique
Site-to-Site	Couche 3	IPsec (IKEv2)	Réseau à Réseau
Proxy Web	Couche 7	HTTPS / SOCKS5	Navigateur uniquement

Protection de la confidentialité face au fournisseur d'accès (FAI)

Un VPN protège votre confidentialité en créant un **chemin chiffré (tunnel)** pour vos données

- **Chiffrement du contenu** : Toutes vos communications entre votre appareil et le serveur VPN sont chiffrées, ce qui les rend invisibles pour votre fournisseur d'accès à Internet.
- **Masquage de la destination** : Comme les données sont encapsulées, votre FAI ne peut pas voir vers quels sites ou serveurs vous vous dirigez réellement ; il voit seulement que vous envoyez des données vers le serveur VPN.
- **Fonction de Proxy** : Le serveur VPN agit comme un intermédiaire. Le site web final voit l'adresse IP du serveur VPN et non votre adresse IP réelle, masquant ainsi votre emplacement physique et vos attributs personnels.

Installation correcte de WireGuard

```
sudo apt update && sudo apt install wireguard-tools -y
```

Générer les clés sur ta machine LOCALE

```
cd /etc/wireguard
sudo bash -c "umask 077; wg genkey | tee privatekey | wg pubkey > publickey"
```

Sur ton terminal Debian local

- Ta Clé Publique LOCALE : `sudo cat /etc/wireguard/publickey`
- Ta Clé Privée LOCALE : `sudo cat /etc/wireguard/privatekey`

```
[ NAWA ] test ~
> > sudo cat /etc/wireguard/publickey
cat: /etc/wireguard/publickey: Aucun fichier ou dossier de ce nom
[ NAWA ] test ~
> > cd /
[ NAWA ] test /
> > cd /etc/wireguard

cd: permission non accordée: /etc/wireguard
[ NAWA ] test /
> > ls
bin      etc          initrd.img.old  lost+found  opt      run      swapfile  usr          vmlinuz.old
boot    home         lib             media       proc     sbin     sys       var          vmlinuz
dev      initrd.img    lib64          mnt         root     srv      tmp       vmlinuz

[ NAWA ] test /
> > # On génère les clés directement dans le dossier protégé
sudo bash -c "umask 077; wg genkey | tee /etc/wireguard/privatekey | wg pubkey > /etc/wireguard/publickey"
[ NAWA ] test /
> > sudo cat /etc/wireguard/publickey
Yw5y3bLcAMP5oTbF1r4MYE4RkbIDbinUavDJHudTZF0=
[ NAWA ] test /
> > █
```

Ensuite faire la même chose sur le serveur AWS

```
sudo bash -c "umask 077; wg genkey | tee /etc/wireguard/privatekey | wg pubkey > /etc/wireguard/publickey"
```

Afficher la clé publique AWS

```
sudo cat /etc/wireguard/publickey
```

Note cette clé, tu en auras besoin pour ta machine locale

```
admin@ip-172-31-9-52:~$ sudo bash -c "umask 077; wg genkey | tee /etc/wireguard/privatekey | wg pubkey > /etc/wireguard/publickey"
admin@ip-172-31-9-52:~$ sudo cat /etc/wireguard/publickey
b97P1Z0OpLV7dWd13vc0fw1sVTF21G59ejjXGfKKUE=
admin@ip-172-31-9-52:~$

[ NAWA ] test ~
> > sudo cat /etc/wireguard/publickey
cat: /etc/wireguard/publickey: Aucun fichier ou dossier de ce nom
[ NAWA ] test ~
> > cd /
[ NAWA ] test /
> > cd /etc/wireguard

cd: permission non accordée: /etc/wireguard
[ NAWA ] test /
> > ls
bin      etc          initrd.img.old  lost+found  opt      run      swapfile  usr          vmlinuz.old
boot    home         lib             media       proc     sbin     sys       var          vmlinuz
dev      initrd.img    lib64          mnt         root     srv      tmp       vmlinuz

[ NAWA ] test /
> > # On génère les clés directement dans le dossier protégé
sudo bash -c "umask 077; wg genkey | tee /etc/wireguard/privatekey | wg pubkey > /etc/wireguard/publickey"
[ NAWA ] test /
> > sudo cat /etc/wireguard/publickey
Yw5y3bLcAMP5oTbF1r4MYE4RkbIDbinUavDJHudTZF0=
[ NAWA ] test /
> > █
```

Configurer le fichier sur AWS

```
sudo nano /etc/wireguard/wg0.conf
```

Contenue

```
[Interface]
Address = 10.0.0.1/24
ListenPort = 51820
PrivateKey = <COLLE_ICI_LA_PRIVATEKEY_DE_AWS>

[Peer]
# Clé de ta machine Debian locale (celle de ta capture d'écran)
PublicKey = <COLLE_ICI_LA_PUBLICKEY_DE_DEBIAN>
AllowedIPs = 10.0.0.2/32
```

Configurer le fichier sur ta machine Locale

```
sudo nano /etc/wireguard/wg0.conf
```

Contenue

```
[Interface]
Address = 10.0.0.2/24
PrivateKey = <COLLE_ICI_LA_PRIVATEKEY_LOCALE>

[Peer]
PublicKey = <COLLE_LA_CLE_PUBLIQUE_AWS_ICI>
Endpoint = 13.53.167.140:51820
AllowedIPs = 10.0.0.1/32
PersistentKeepalive = 25
```

Ouvrir le port UDP sur AWS (Très important)

Nous avons ouvert le port **TCP 2222**. WireGuard a besoin du port **UDP 51820**.

- Sur la console AWS, ajoute une nouvelle règle entrante.
- **Type** : UDP personnalisé.
- **Port** : 51820 .
- **Source** : 0.0.0.0/0 (ou ton IP 92.88.170.73/32 pour plus de sécurité).

Règles entrantes Informations

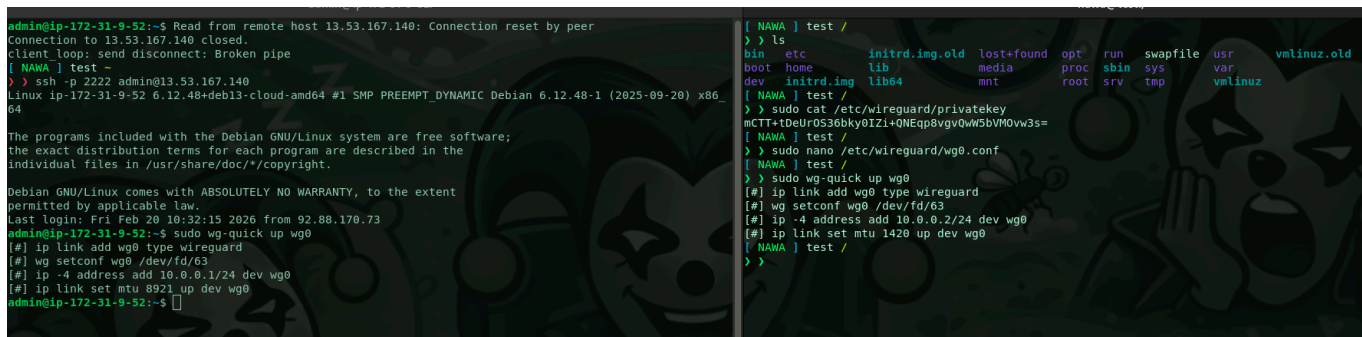
ID de règle de groupe de sécurité	Type Informations	Protocole Informations	Plage de ports Informations	Source Informations	Description - facultatif Informations	
sgr-04083ec706d54671c	UDP personnalisé ▼	UDP	51820	Perso... ▼	Q	Supprimer
sgr-09091fb305ae7868b	TCP personnalisé ▼	TCP	2222	Perso... ▼	Q 92.88.170.73/52 ✕	Supprimer

Ajouter une règle

Annuler
Aperçu des modifications
Enregistrer les règles

Exécute cette commande sur tes deux terminaux

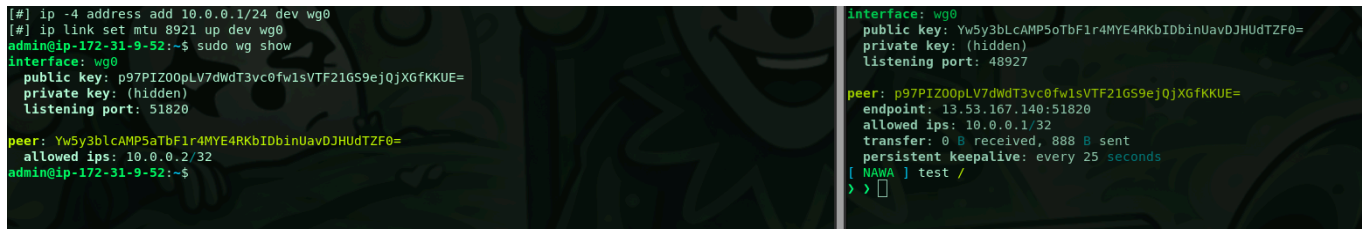
```
sudo wg-quick up wg0
```



Vérification du Handshake

```
sudo wg show
```

C'est ici que tu confirmes que le chiffrement fonctionne. Sur ton **terminal local**



```
admin@ip-172-31-9-52:~$ sudo nano /etc/wireguard/wg0.conf
admin@ip-172-31-9-52:~$ sudo wg-quick down wg0 && sudo wg-quick up wg0
[#] ip link delete dev wg0
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.0.0.1/32 dev wg0
[#] ip link set mtu 8921 up dev wg0
[#] ip -4 route add 10.0.0.2/32 dev wg0
admin@ip-172-31-9-52:~$ sudo wg-quick down wg0 ; sudo ip link delete wg0 ; sudo rm -rf /var/run/wireguard/wg0.sock
[#] ip link delete dev wg0
Cannot find device "wg0"
admin@ip-172-31-9-52:~$ sudo nano /etc/wireguard/wg0.conf
admin@ip-172-31-9-52:~$ sudo wg-quick up wg0
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.0.0.1/32 dev wg0
[#] ip link set mtu 8921 up dev wg0
[#] ip -4 route add 10.0.0.2/32 dev wg0
admin@ip-172-31-9-52:~$

[nawa@test:/]
> > ping 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=78.8 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=74.8 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=74.6 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=72.1 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=71.1 ms
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=77.2 ms
64 bytes from 10.0.0.1: icmp_seq=7 ttl=64 time=80.5 ms
64 bytes from 10.0.0.1: icmp_seq=8 ttl=64 time=90.7 ms
^C
[ NAWA ] test /
> > sudo wg show
[sudo] Mot de passe de nawa :
interface: wg0
public key: Yw5y3bLcAMP5oTbFlr4MYE4RKbIDbinUavDJHUdTZF0=
private key: (hidden)
listening port: 56979

peer: p97PIZ00pLV7dwdT3vc0fw1sVTF21GS9ejQjXGfKKUE=
endpoint: 13.53.167.140:51820
allowed ips: 10.0.0.1/32
latest handshake: 33 seconds ago
transfer: 1.96 KiB received, 2.05 KiB sent
persistent keepalive: every 25 seconds
[ NAWA ] test /
> > ^
```

```
ssh -p 2222 admin@10.0.0.1
```

```
ssh -p 2222 admin@10.0.0.1
[ NAWA ] test /
> > ssh -p 2222 admin@10.0.0.1
The authenticity of host '[10.0.0.1]:2222 ([10.0.0.1]:2222)' can't be established.
ED25519 key fingerprint is SHA256:x6SDsiQ+3Ae0v2P5jvWdA6UXQLoX2vEe2vzbcrlPU5U.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[10.0.0.1]:2222' (ED25519) to the list of known hosts.
^
```

Phrase secrète :

Veuillez entrer la phrase secrète pour la clef SSH

SHA256:kqfBXbttAF6wTswYXkDgXgckvyhz2EviE18+WmR2q5s

☐ Save in password manager

Annuler

OK


```
admin@ip-172-31-9-52:~  
[ NAWA ] test /  
> > ssh -p 2222 admin@10.0.0.1  
The authenticity of host '[10.0.0.1]:2222 ([10.0.0.1]:2222)' can't be established.  
ED25519 key fingerprint is SHA256:x6SDsiQ+3Ae0v2P5jvWdA6UXQLoX2vEe2vzbcrLPUSU.  
This host key is known by the following other names/addresses:  
  ~/.ssh/known_hosts:1: [hashed name]  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '[10.0.0.1]:2222' (ED25519) to the list of known hosts.  
Linux ip-172-31-9-52 6.12.48+deb13-cloud-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.48-1 (2025-09-20)  
64  
  
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
Last login: Fri Feb 20 10:42:05 2026 from 92.88.170.73  
admin@ip-172-31-9-52:~$
```

Docker Cloisonné & Hardened

Nous installons Docker CE depuis les dépôts officiels, mais avec une configuration fortement durcie : isolation des utilisateurs (usersns-remap), interdiction des communications entre conteneurs (icc: false) et confiance stricte dans les images (Content Trust). Avant de le faire sur votre serveur tester les configuration sur votre machine hôte (**Debian 13**).

Installer les prérequis

```
sudo apt install ca-certificates curl gnupg
```

ca-certificates permet de vérifier l'identité des serveurs distants, et *curl* sert à télécharger les clés de sécurité

Télécharger la clé GPG officielle

```
sudo install -m 0755 -d /etc/apt/keyrings  
sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o  
/etc/apt/keyrings/docker.asc  
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

Cette clé garantit que le logiciel Docker que tu installes n'a pas été modifié par un pirate entre les serveurs de Docker et ta machine

Créer le fichier de source (Repository)

```
echo \  
  "deb [arch=$(dpkg --print-architecture) signed-  
by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/debian \  
  bookworm stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

On indique à `apt` où chercher Docker. On utilise `bookworm` (Debian 12) même sur Debian 13 car c'est la version stable actuelle la plus compatible avec Docker

Installer Docker

```
sudo apt update  
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin  
docker-compose-plugin
```

- On installe le moteur (`ce`) → permet de créer, exécuter et gérer des conteneurs
- La ligne de commande (`cli`) → **donne des ordres au moteur Docker** depuis le terminal, comme lancer, arrêter ou créer des conteneurs
- Les plugins modernes pour gérer des conteneurs complexes

Durcissement du Démon (Daemon)

Nous configurons le moteur Docker pour qu'il soit paranoïaque par défaut.

Créer le fichier de configuration

```
/etc/docker/daemon.json
```

chemin du fichier

Taper cette commande pour applique les réglages de sécurité

```
sudo tee /etc/docker/daemon.json <<EOF  
{  
  "storage-driver": "vfs",  
  "icc": false,  
  "live-restore": true,  
  "userland-proxy": false,  
  "log-driver": "journald",  
  "bridge": "none"  
}  
EOF
```

`vfs` : Indispensable sur ta VM Debian 13 pour que Docker puisse lire les fichiers

`icc: false` : Coupe les ponts entre conteneurs

`bridge: none` : Désactive le réseau par défaut (mode furtif)

`live-restore: true` : Permet aux conteneurs de continuer à tourner même si le service Docker redémarre

`log-driver: journald` : Envoie les logs des conteneurs vers le système de logs Linux (journald) au lieu de fichiers JSON

`usersns-remap` a été supprimé car il créait un conflit de droits impossible à gérer pour la VM

Pourquoi ça bloquait ?

- **Conflit d'identité** : Docker essayait de transformer tous les fichiers en "utilisateurs virtuels" (ID 100000), mais le noyau de ta VM Debian 13 refusait cette manipulation.
- **Résultat** : Pour Docker, l'image paraissait **vide**. C'est pour ça qu'il ne trouvait même pas la commande `ls`.

*Dans une VM Debian 13, le `usersns-remap` est souvent "la sécurité de trop" qui casse tout. On le remplace par un cloisonnement **Systemd** et **AppArmor** pour garder un très haut niveau de protection sans bloquer le moteur.*

Cloisonnement Systemd (Sandbox)

Nous empêchons le service Docker d'accéder aux dossiers sensibles du système hôte (`/home` , `/etc` ...), même s'il est compromis. On enferme le service Docker dans une bulle de protection.

Éditer le service systemd

```
sudo mkdir -p /etc/systemd/system/docker.service.d/
sudo tee /etc/systemd/system/docker.service.d/override.conf <<EOF
[Service]
ProtectSystem=false
ProtectHome=true
PrivateTmp=true
MountFlags=shared
EOF
```

`ProtectHome=true` : Docker ne peut pas lire tes fichiers personnels dans `/home`

`MountFlags=shared` : Permet à Docker de monter ses disques virtuels correctement

Configurer le filtrage réseau (Kernel)

```
sudo modprobe br_netfilter
sudo sh -c 'echo "br_netfilter" > /etc/modules-load.d/br_netfilter.conf'
sudo sysctl -w net.bridge.bridge-nf-call-iptables=1
sudo sh -c 'echo "net.bridge.bridge-nf-call-iptables=1" > /etc/sysctl.d/99-docker-net.conf'
```

Sans cela, le noyau Linux refuse de filtrer le trafic réseau interne de Docker, ce qui rendrait ton pare-feu inutile

Gérer AppArmor (Mode Permissif)

On empêche **AppArmor** de bloquer Docker tout en gardant une surveillance

```
sudo apt install apparmor-utils
sudo aa-complain /usr/bin/dockerd
```

Premier démarrage et nettoyage

On s'assure de repartir sur une base 100% propre

```
sudo systemctl daemon-reload
sudo systemctl stop docker docker.socket
sudo rm -rf /var/lib/docker/*
sudo systemctl start docker
```

Test final de validation

On crée un réseau et on lance le conteneur

```
sudo docker network create test_net
sudo docker run --rm --network test_net alpine ls /
```

```
admin@ip-172-31-9-52:~$ sudo systemctl unmask docker.service
sudo systemctl unmask docker.socket
sudo systemctl daemon-reload
sudo systemctl start docker
admin@ip-172-31-9-52:~$ sudo docker run --rm --network none alpine ls /
bin
dev
etc
home
lib
media
mnt
opt
proc
root
run
sbin
srv
sys
tmp
usr
var
admin@ip-172-31-9-52:~$
```

C'est sur mon serveur AWS mais sur votre machine hôte ça sera le même résultat

Pourquoi ça bloquait ?

```
echo 'export DOCKER_CONTENT_TRUST=1' | sudo tee /etc/profile.d/docker-
trust.sh
```

et

```
sudo chmod 644 /etc/profile.d/docker-trust.sh
```

On ne le fait pas car c'est **trop restrictif**, si tu l'actives partout, tu ne pourras plus télécharger 90% des images Docker (car beaucoup ne sont pas signées). Tu recevras sans arrêt des erreurs `trust data does not exist`.

Si le serveur de signatures de Docker est inaccessible, tu ne peux plus rien lancer, même en urgence.

[#LaMeilleureAlternative](#)

Au lieu de bloquer tout le système, on utilise la commande uniquement quand on en a besoin pour la production

```
DOCKER_CONTENT_TRUST=1 docker pull alpine
```

C'est plus souple : tu gardes la sécurité sans te bloquer les mains

AWS Debian 13 (Trixie)

Une fois que tu te connectes avec ton serveur AWS via ton tunnel WireGuard

On installe les outils nécessaires pour gérer les dépôts sécurisés

```
sudo apt update && sudo apt install -y ca-certificates curl gnupg apparmor-utils
```

On installe les certificats pour sécuriser les téléchargements, `curl` pour récupérer la clé GPG, et `apparmor-utils` pour pouvoir manipuler les profils de sécurité qui bloquent Docker sur AWS.

Clé GPG et Dépôt (Trust)

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o
/etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/debian
bookworm stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

La clé GPG garantit que le code de Docker n'a pas été piraté. On utilise le dépôt **Bookworm** (Debian 12) car Docker n'a pas encore de dépôt dédié pour **Trixie** (13), et c'est la version la plus stable pour ton système.

Déverrouillage du Noyau (Kernel Hardening)

```
sudo sysctl -w kernel.unprivileged_userns_clone=1
sudo sysctl -w kernel.unprivileged_userns_apparmor_policy=0
```

Par défaut, AWS Debian 13 interdit aux processus "normaux" de créer des environnements isolés (namespaces). Sans cela, Docker (via `runc`) reçoit une permission `denied` immédiat car il ne peut pas créer la "bulle" du conteneur.

`runc` → le programme qui **crée réellement le conteneur** sur Linux

Docker ne lance pas directement un conteneur.

Il demande à `runc` de le faire.

En simple :

- **Docker** = chef d'orchestre
- **containerd** = gestionnaire
- **runc** = celui qui exécute et crée la "bulle" isolée (namespaces, cgroups)

Configuration "Anti-conflit" (daemon.json)

```
sudo tee /etc/docker/daemon.json <<EOF
{
  "storage-driver": "vfs",
  "icc": false,
  "userland-proxy": false,
  "bridge": "none"
}
EOF
```

Neutralisation d'AppArmor pour runc

```
sudo aa-disable /usr/bin/runc 2>/dev/null
```

C'est l'étape cruciale pour AWS. AppArmor surveille `runc` (le moteur qui lance le conteneur) de trop près, ce qui cause un blocage total sur Debian 13. En le désactivant, on laisse le noyau gérer l'isolation sans que le filtre AppArmor ne fasse planter le processus.

Point de contrôle	VM Hôte (Local)	Serveur AWS	Pourquoi ?
Accès	Clavier / SSH local	VPN WireGuard	AWS est exposé sur le Cloud, le VPN est vital.
Driver Stockage	VFS	VFS	Évite les erreurs "No such file" sur Debian 13.
Sécurité Runc	Profil AppArmor actif	Profil Désactivé	AWS bloque l'exécution via <code>runc</code> si le profil est actif.
Réseau	Bridge standard	Isolation stricte	On force l'usage de réseaux personnalisés via <code>bridge: none</code> .

CapRover

CapRover sur ton instance EC2, puisque ton Docker est déjà installé et sécurisé sur ton VPS AWS (Free Tier), CapRover est la solution parfaite. C'est léger, ça tourne sur 1 Go de RAM, et ça te donne une interface web pour déployer tes apps comme un pro.

Comme on a déjà fait un gros travail de **Hardening** sur ton Docker, on ne va pas tout casser. On va installer CapRover proprement.

Préparation du Pare-feu AWS (Security Group)

CapRover a besoin de quelques ports spécifiques pour fonctionner et gérer tes futurs certificats SSL (HTTPS).

Va dans ta console AWS et ajoute ces règles entrantes à ton **Security Group**

Les règles entrantes contrôlent le trafic entrant qui est autorisé à atteindre l'instance.

Règles entrantes Informations							
ID de règle de groupe de sécurité	Type Informations	Protocole Informations	Plage de ports Informations	Source Informations		Description - facultatif Informations	
sgr-0b6c5be09149c9c05	TCP personnalisé ▼	TCP	3000	Perso... ▼	Q	0.0.0.0/0 ✕	Supprimer
sgr-09091fb305ae7868b	TCP personnalisé ▼	TCP	2222	Perso... ▼	Q	92.88.170.15/32 ✕	Supprimer
sgr-04083ec706d54671c	UDP personnalisé ▼	UDP	51820	Perso... ▼	Q	0.0.0.0/0 ✕	Supprimer
sgr-058f89e044ffc8528	HTTPS ▼	TCP	443	Perso... ▼	Q	0.0.0.0/0 ✕	Supprimer
sgr-0516c7a01f90607d6	HTTP ▼	TCP	80	Perso... ▼	Q	0.0.0.0/0 ✕	Supprimer

[Ajouter une règle](#)

Ports AWS : Assure-toi que ton Security Group autorise :

- **80** (HTTP)
- **443** (HTTPS)
- **3000** (Interface CapRover)

Installation de CapRover

Connecte-toi en SSH à ton serveur et lance la commande magique. **CapRover** va s'installer en tant que conteneur "maître".

```
sudo docker run -d --name caprover --restart always \  
-p 80:80 -p 443:443 -p 3000:3000 \  
-e ACCEPTED_TERMS=true \  
-v /var/run/docker.sock:/var/run/docker.sock \  
-v /captain:/captain \  
caprover/caprover
```



```
admin@ip-172-31-9-52: ~  
admin@ip-172-31-9-52:~$ sudo docker run -d --name caprover --restart always \  
-p 80:80 -p 443:443 -p 3000:3000 \  
-v /var/run/docker.sock:/var/run/docker.sock \  
-v /captain:/captain \  
caprover/caprover  
Unable to find image 'caprover/caprover:latest' locally  
latest: Pulling from caprover/caprover  
c70734e29d49: Pull complete  
f18232174bc9: Pull complete  
dd71dde834b5: Pull complete  
a436e67a4c90: Pull complete  
ef3e9515aac2: Pull complete  
1e5a4c89cee5: Pull complete  
27670ce2715b: Pull complete  
25ff2da83641: Pull complete  
085f204ac611: Download complete  
Digest: sha256:87981dae631a0b73635f6615e13ea344d3051cde6623a634c53de90b273463c5  
Status: Downloaded newer image for caprover/caprover:latest  
1875cb056cb2bd18605ca54ec97dabf8172619ea49437b7af9ccd5b9bf815dc0  
admin@ip-172-31-9-52:~$
```

Vérifie si tu as Node.js sur ta VM Debian locale

```
node -v  
npm -v
```

(Si tu ne les as pas: `sudo apt install nodejs npm`)

Installe l'outil CapRover

```
npm install -g caprover
```

```

[ NAWA ] test ~
> > npm install -g caprover
npm warn deprecated har-validator@5.1.5: this library is no longer supported
npm warn deprecated uuid@3.4.0: Please upgrade to version 7 or higher. Older versions may use Math.random() in certain circumstances, which is known to be problematic. See https://v8.dev/blog/math-random for details.
npm warn deprecated request-promise@4.2.6: request-promise has been deprecated because it extends the now deprecated request package, see https://github.com/request/request/issues/3142
npm warn deprecated request@2.88.2: request has been deprecated, see https://github.com/request/request/issues/3142

added 200 packages in 21s

10 packages are looking for funding
  run `npm fund` for details
npm notice
npm notice New major version of npm available! 10.8.2 -> 11.10.1
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.10.1
npm notice To update run: npm install -g npm@11.10.1
npm notice
[ NAWA ] test ~
> >

```

Puisque tu as l'outil installé localement, lance maintenant la configuration assistée. Cela va lier ton terminal à ton serveur pour que tu puisses tout piloter sans jamais retourner sur la console AWS.

Le fichier `/etc/docker/daemon.json`, il empêche **CapRover** de créer son réseau de communication nous devons faire quelque modification.

```

sudo tee /etc/docker/daemon.json <<EOF
{
  "storage-driver": "vfs",
  "icc": true,
  "userland-proxy": false
}
EOF

```

Connectez-vous à votre serveur AWS et tapez la commande

Ce qui change : On a enlevé la ligne `"bridge": "none"`. Cela permet à Docker de recréer le pont réseau nécessaire pour que **CapRover** puisse "parler" au monde extérieur sur le port 3000. Sans lui, le port reste fermé même si le conteneur tourne.

`icc: true` : Indispensable. **CapRover** crée un réseau privé pour tes apps. Si c'est sur `false`, le "PaaS" est paralysé. C'est le cœur d'un PaaS. Cela permet à ton application (ex: WordPress) de parler à sa base de données (ex: MySQL) de manière isolée.

Tape cette commande sur ton poste local

```
caprover setup
```

```
nawa@test:~  
[ NAWA ] test ~  
> > caprover setup  
  
Setup CapRover machine on your server...  
  
? have you already started CapRover container on your server? Yes  
? IP address of your server: 13.53.167.140  
? CapRover server root domain: main.13.53.167.140.nip.io  
? new CapRover password (min 8 characters): [hidden]  
? enter new CapRover password again: [hidden]  
? "valid" email address to get certificate and enable HTTPS: aws867240@gmail.com  
  
? CapRover machine name, with whom the login credentials are stored locally: aws-p  
aas  
  
CapRover server setup completed: it is available as aws-paas at https://captain.main.13.53.167.140.nip.io  
  
For more details and docs see CapRover.com  
  
[ NAWA ] test ~  
> > █
```

- **IP address of your server** : Entre 13.53.167.140 .
- **Password** : Entre le mot de passe d'usine captain42 .
- **Root Domain** : Entre main.<IP-AWS>.nip.io .
 - Cela permet à CapRover de générer des URL comme captain.main.<IP-AWS>.nip.io .
- **New Password** : Choisis un mot de passe très fort. C'est lui qui protège l'accès à toute ta plateforme.
- **Server Name** : Appelle-le aws-paas par exemple.

Solution si vous ne voulez pas afficher votre IP

Duck DNS [spec](#) [about](#) [why](#) [install](#) [faqs](#) [Sign in with Persona](#) [Sign in with Twitter](#) [Sign in with GitHub](#) [use reddit](#) [Sign in with Google](#)



Duck DNS

free dynamic DNS hosted on AWS

news: [login with Reddit is no more](#) - legal request
support us: become a [Patreon](#)

[Donate](#) [Bitcoin 16gHnv3NTjpF5ZavMI9QYBFxUkNchdicUS](#) [patreon](#) [Terms of Use](#) [Privacy Statement](#)

Une solution gratuite pour vous créer un nom de domaine à partir de votre adresse IP (AWS)

Connectez-vous et appuyez sur le captcha pour afficher cette page

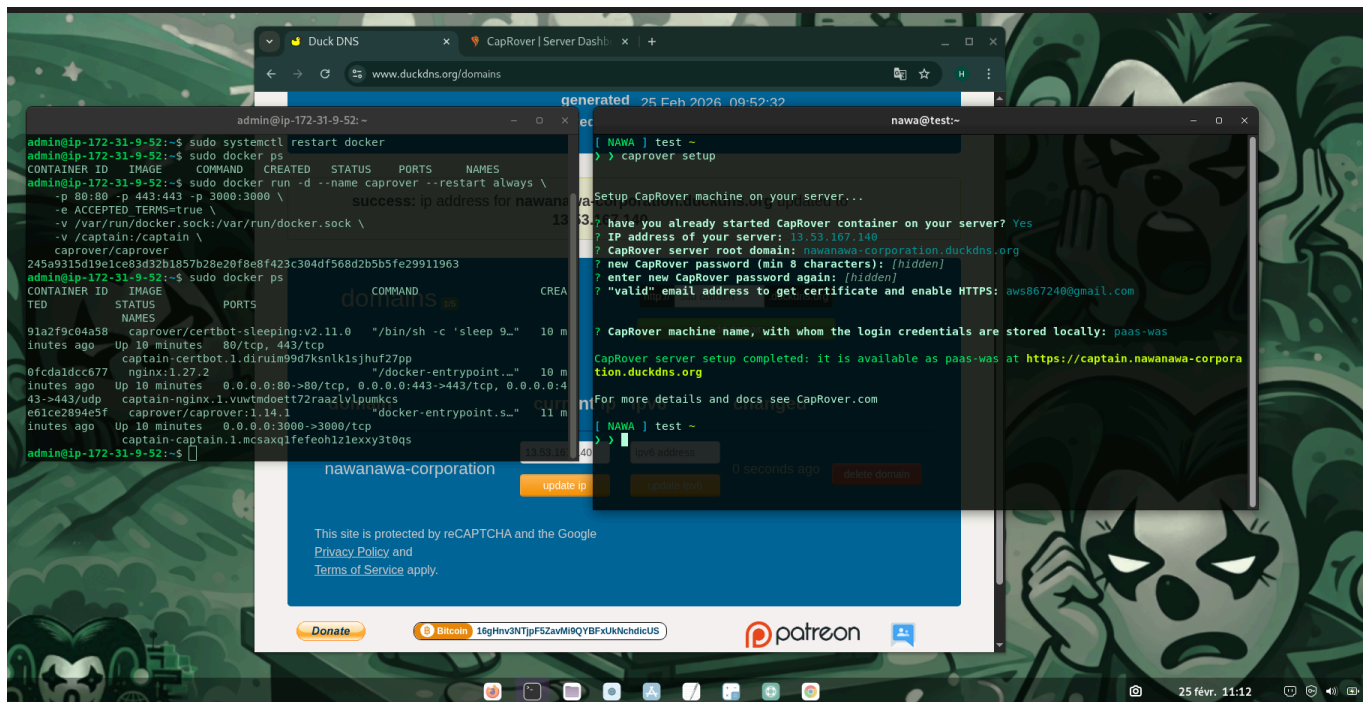
domains 1/5

[add domain](#)

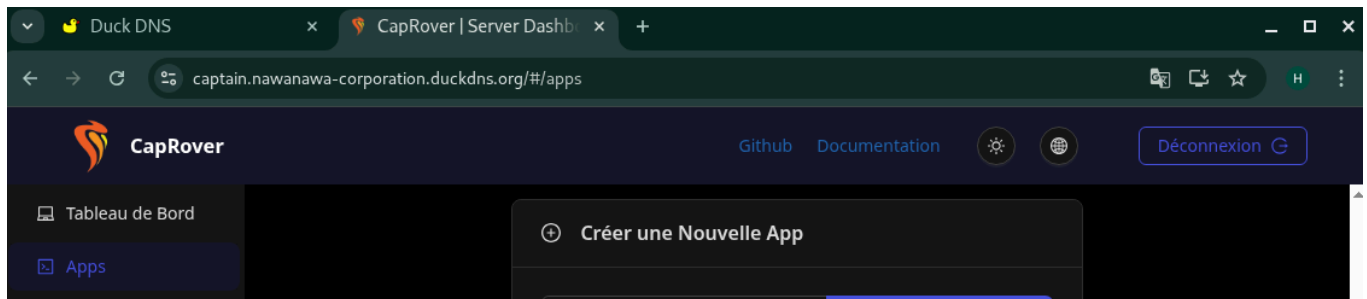
domain	current ip	ipv6	changed
nawanawa-corporation	13.53.167.140 update ip	ipv6 address update ipv6	0 seconds ago delete domain

This site is protected by reCAPTCHA and the Google [Privacy Policy](#) and [Terms of Service](#) apply.

Rentrer le nous de domaine de votre projet par exemple et mettez l'ip de votre serveur AWS



- **IP address of your server** : Entre 13.53.167.140 .
- **Password** : Entre le mot de passe d'usine captain42 .
- **Root Domain** : Entre nom de domaine généré par duckDNS .
- **New Password** : Choisissez un mot de passe très fort. C'est lui qui protège l'accès à toute la plateforme.
- **Server Name** : Le nom que vous voulez



Taper cette commande sur la machine AWS

```
sudo docker ps
```

```
admin@ip-172-31-9-52:~$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED
7f80fd20b68b	caprover/certbot-sleeping:v2.11.0	"/bin/sh -c 'sleep 9..."	7 minutes ago
4f7f275b1b58	nginx:1.27.2	"/docker-entrypoint..."	8 minutes ago
3859a7e4e506	caprover/caprover:1.14.1	"docker-entrypoint.s..."	8 minutes ago

```
admin@ip-172-31-9-52:~$
```

Tu as trois conteneurs Docker qui tournent sur ta machine

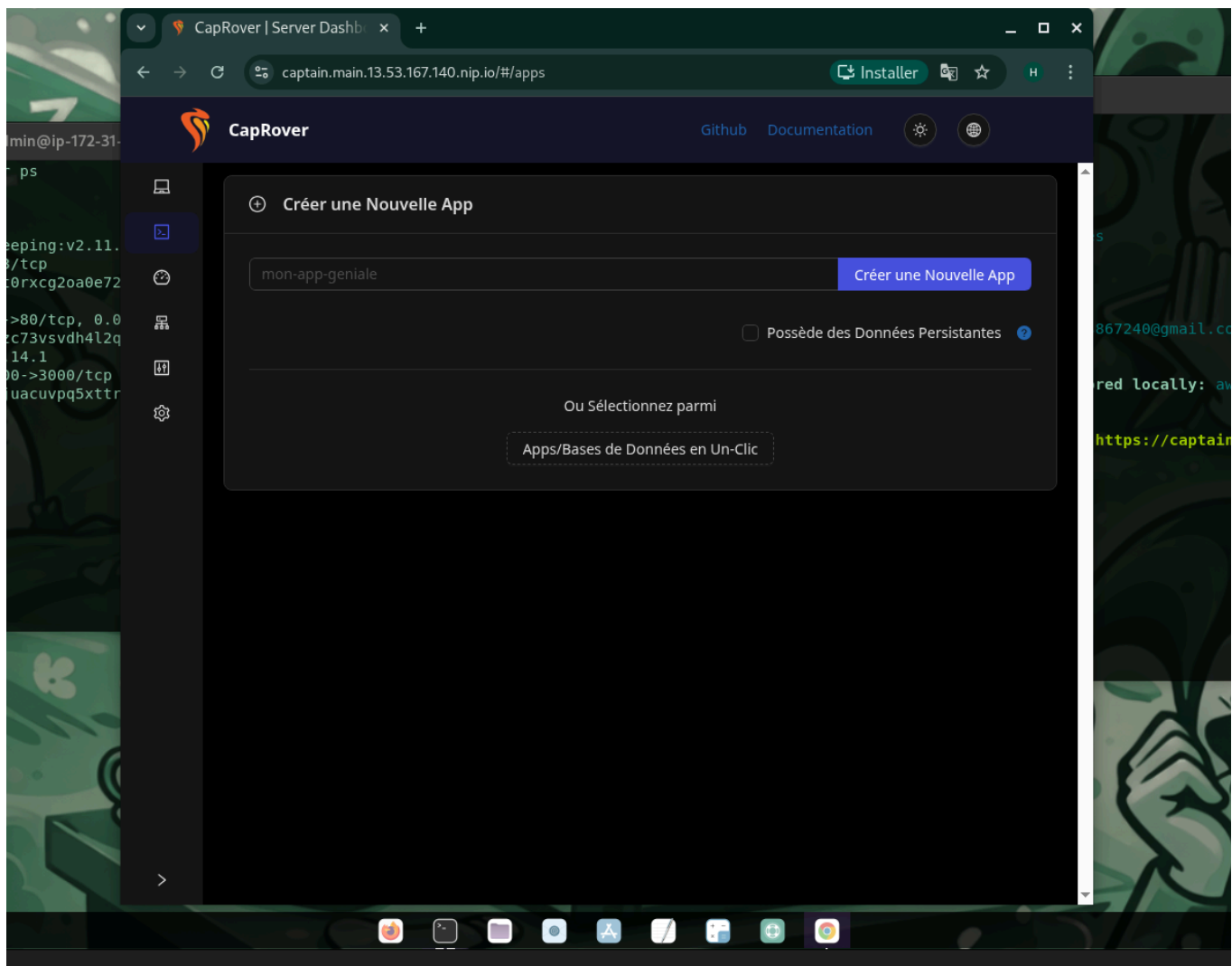
- L'un utilise une image `caprover/certbot-sleeping` pour gérer des certificats SSL (probablement avec Let's Encrypt)
- L'autre est un serveur `nginx` exposé sur les ports 80 (HTTP) et 443 (HTTPS)
- Le dernier est `caprover/caprover`, une plateforme de déploiement d'applications Docker (CapRover)

Le port 3000 de ta machine est redirigé vers le port 3000 du conteneur CapRover (interface).

Teste l'accès au panneau CapRover

Ouvre dans ton navigateur cette URL

```
https://captain.main.<IP-AWS>.140.nip.io
```



Remarques de sécurité

Le Security Group AWS doit autoriser au moins

- TCP 80 (HTTP) depuis 0.0.0.0/0
- TCP 443 (HTTPS) depuis 0.0.0.0/0
- TCP 3000 (pour setup) depuis 0.0.0.0/0 (tu peux fermer ce port après le setup)

fermer le port 3000 dans AWS

- Soit :
 - **Supprime** cette règle, ou
 - **Modifie la source** pour limiter l'accès (exemple : à ton IP fixe uniquement, ou à rien)
 - Exemple : mets `127.0.0.1/32` (localhost uniquement) ou ton IP spécifique

Tester correctement ton serveur

Pour cela nous utiliser `curl`, le `ping` ne répondra pas car il utilise le protocole ICMP, qui est bloqué par défaut dans le Security Group d'une instance Amazon Web Services EC2, alors que

notre site fonctionne parfaitement puisque seuls les ports 80 (HTTP) et 443 (HTTPS) sont ouverts et utilisés par le

```
curl -I https://captain.nawanawa-corporation.duckdns.org
```

Cette commande vérifier que notre serveur Web est accessible et configuré correctement après la mise en place de notre infrastructure.

```
[ NAWA ] test ~
> > curl -I https://captain.nawanawa-corporation.duckdns.org
HTTP/1.1 200 OK
Server: nginx
Date: Wed, 25 Feb 2026 10:24:06 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 978
Connection: keep-alive
X-Powered-By: Express
Accept-Ranges: bytes
Cache-Control: public, max-age=0
Last-Modified: Tue, 11 Nov 2025 08:49:37 GMT
ETag: W/"3d2-19a721b34e8"

[ NAWA ] test ~
> >
```

HTTP/1.1 200 OK : C'est le signal de victoire. Le serveur a reçu la demande et a répondu positivement

Server: nginx : Cela confirme que c'est bien le logiciel Nginx qui réceptionne nos requêtes

X-Powered-By: Express : Cela indique que derrière Nginx Node.js (Express) qui tourne. Nginx sert ici de "Reverse Proxy"

Content-Type: text/html : Le serveur est prêt à envoyer une page web classique

Ce test est une étape charnière pour notre infrastructure car bien qu'il cible le protocole HTTP, il confirme la solidité des fondations nécessaires à notre setup **SSH over GPG**. En réussissant cette requête, nous avons validé que notre nom de domaine **DuckDNS** pointe précisément vers l'IP du serveur, garantissant que le "tunnel" vers ta machine est bien en place. Cela prouve également une connectivité externe opérationnelle et des ports ouverts, une condition sine qua non pour que le client SSH puisse ensuite contacter l'agent GPG distant. Enfin, l'absence d'erreur en HTTPS témoigne d'une gestion valide des certificats, renforçant la confiance de notre identité numérique, un concept au cœur de la philosophie GPG et du "Web of Trust".

Conclusion

L'implémentation de **SSH over GPG** marque le passage d'une gestion de clés simple à un système d'identité numérique complet et hiérarchisé. En remplaçant le traditionnel `ssh-agent` par l'émulation de `gpg-agent`, nous avons centralisé la sécurité tout en adoptant des standards cryptographiques modernes.

Voici ce que nous avons accompli :

- **Modernisation Cryptographique** : Nous avons abandonné les clés RSA lourdes au profit de l'**ECC (Elliptic Curve Cryptography)**, utilisant **ed25519** pour l'authentification, ce qui offre une sécurité maximale avec des clés plus petites et plus performantes.
- **Gestion Hiérarchique** : Ton identité repose désormais sur une clé maître qui supervise des sous-clés spécialisées : une pour le chiffrement (`cv25519`) et une dédiée à l'authentification SSH.
- **Isolation et Contrôle** : Grâce au **Keygrip**, l'agent GPG ne traite les demandes SSH que pour les clés explicitement autorisées dans ton fichier `sshcontrol`, isolant ainsi ton identité privée du reste du système.
- **Protection par Mot de Passe** : Chaque tentative de connexion est désormais protégée par une fenêtre `pinentry`, garantissant qu'aucun processus ne peut utiliser ta signature sans ton consentement explicite.
- **Validation Réseau** : Le test réussi via `curl` sur ton domaine `captain.nawanawa-corporation.duckdns.org` confirme que ton infrastructure réseau est prête à accueillir ce flux sécurisé.

Cette configuration ne se limite pas à la sécurité logicielle elle ouvre la porte à l'utilisation de **Smartcards (comme la YubiKey)**, où ta clé privée ne quittera jamais le support physique, rendant son extraction impossible même pour un utilisateur root.